

hw1

TING-YU CHANG

This is the link to my repo on github:<https://github.com/tingyuuucc/stats506-HW1>

Q1: Exploratory Analysis of Soybean Cultivars

a. Import the data into a data.frame in R. Use the dataset documentation to assign appropriate variable names. (Note: Downloading and unzipping the file can take place outside of your submitted document, but importing the file should be in the submission.)

```
soybean <- read.csv("data.csv")

colnames(soybean) <- c(
  "Season",
  "Cultivar",
  "Repetition",
  "PH",
  "IFP",
  "NLP",
  "NGP",
  "NGL",
  "NS",
  "MHG",
  "GY"
)
```

b. Using the information provided in the dataset documentation: Verify that there are 320 observations in the dataset; 40 distinct cultivars; and all the observations were collected from two seasons. Produce a table showing the number of observations from each season.

```
# number of obs.  
nrow(soybean)
```

```
[1] 320
```

```
# number of distinct cultivars  
length(unique(soybean$Cultivar))
```

```
[1] 40
```

```
# number of distinct seasons  
length(unique(soybean$Season))
```

```
[1] 2
```

```
# number of observations from each season  
table(soybean$Season)
```

```
 1  2  
160 160
```

The dataset contains 320 observations, 40 distinct cultivars, and two seasons. Each season contains 160 observations.

c. Use the data to answer the following questions

1. What is the correlation between plant height (PH) and grain yield (GY)?

```
cor(soybean$PH, soybean$GY)
```

```
[1] 0.1232807
```

The correlation between plant height (PH) and grain yield (GY) is 0.1232807.

2. Calculate the correlation between PH and GY separately for each season. Which season has the higher correlation?

```
cor(soybean$PH[soybean$Season==1],  
    soybean$GY[soybean$Season==1])
```

```
[1] 0.2308581
```

```
cor(soybean$PH[soybean$Season==2],  
    soybean$GY[soybean$Season==2])
```

```
[1] -0.0227046
```

Season 2 has the higher correlation.

3. What is the grain yield of the soybean observation with the greatest plant height?

```
soybean[soybean$PH == max(soybean$PH), c("PH", "GY")]
```

	PH	GY
315	94.8	3304.678

The grain yield of the soybean observation with the greatest plant height = 3304.678.

4. What percentage of observations have a thousand seed weight (MHG) greater than the overall average thousand seed weight?

```
mean_MHG <- mean(soybean$MHG)  
# covert to %  
mean(soybean$MHG > mean_MHG) * 100
```

```
[1] 47.1875
```

47.1875%

5. Which cultivar has the highest average grain yield?

```
# calculate average GY
avg_GY <- aggregate(GY ~ Cultivar,
                     data = soybean,
                     FUN = mean)
# find the highest average GY
avg_GY[which.max(avg_GY$GY), ]
```

```
  Cultivar      GY
9 98R30 CE 3964.968
```

The answer is 98R30 CE.

d. Create a table showing the average value of each quantitative variable: One row containing overall averages. One row for each season containing season-specific averages. The table should clearly identify what each row represents. (This table does not need to be “fancy” but should clearly identify what each value represents.)

```
# quantitative variables
quant_vars <- c("PH", "IFP", "NGP", "NGL", "NS", "MHG", "GY")

# overall averages
overall_avg <- data.frame(
  Group = "Overall",
  t(colMeans(soybean[quant_vars]))
)

# season-specific averages
season_avg <- aggregate(
  soybean[quant_vars],
  by = list(Group = paste0("Season ", soybean$Season)),
  FUN = mean
)

# ChatGPT helped me figure out a more efficient way to code my loop for problem d by doing a
#combine into one table
avg_table <- rbind(overall_avg, season_avg)
```

```
avg_table
```

	Group	PH	IFP	NGP	NGL	NS	MHG	GY
1	Overall	68.38678	15.465	135.0858	2.290844	4.071656	168.3223	3418.554
2	Season 1	65.98506	16.485	135.2350	2.299688	4.816625	162.2888	3428.452
3	Season 2	70.78850	14.445	134.9367	2.282000	3.326687	174.3558	3408.655

e. Carry out a series of t-tests to examine whether the ngrain yield (GY) differs between the two seasons. State the null and the alternative hypotheses. Present the R output and interpret the results.

```
# calculate GY by season
GY1 <- soybean$GY[soybean$Season == 1]
GY2 <- soybean$GY[soybean$Season == 2]

# sample statistics
n1 <- length(GY1)
n2 <- length(GY2)

mean1 <- mean(GY1)
mean2 <- mean(GY2)

var1 <- var(GY1)
var2 <- var(GY2)

# calculate t statistics
t_stat <- (mean1 - mean2) / sqrt(var1 / n1 + var2 / n2)

# calculate df
df <- (var1 / n1 + var2 / n2)^2 / ((var1 / n1)^2 / (n1 - 1) + (var2 / n2)^2 / (n2 - 1))

# calculate two-sided p-value
p_value <- 2 * pt(-abs(t_stat), df = df)

t_stat
```

```
[1] 0.351546
```

```
df
```

```
[1] 231.8466
```

```
p_value
```

```
[1] 0.725498
```

```
t.test(GY ~ Season, data = soybean)
```

Welch Two Sample t-test

data: GY by Season

t = 0.35155, df = 231.85, p-value = 0.7255

alternative hypothesis: true difference in means between group 1 and group 2 is not equal to

95 percent confidence interval:

-91.15712 130.75170

sample estimates:

mean in group 1 mean in group 2

3428.452 3408.655

Null hypothesis: $H_0 : \mu_1 = \mu_2$, meaning that the mean grain yield is the same in the two seasons. Alternative hypothesis: $H_0 : \mu_1 \neq \mu_2$, meaning that the mean grain yield differs between the two seasons. The Welch two-sample t-test gave ($t = 0.35155$), ($df = 231.85$), and a p-value of 0.7255. Since the p-value is greater than 0.05, we fail to reject the null hypothesis. There is not sufficient evidence to conclude that the mean grain yield differs between the two seasons.

ChatGPT helped me figure out a clearer way to show my result of e by using latex.

Q2:Adult Income Dataset

a. Import the data into a dataframe in R. As with the previous data, you may download the data outside of your submission, but importation should take place inside the problem set submission. B

```

# import the dataset
adult <- read.csv(
  "adult/adult.data",
  header = FALSE,
  strip.white = TRUE,
  na.string = "?"
)

# assign variable name
colnames(adult) <- c(
  "age",
  "workclass",
  "fnlwgt",
  "education",
  "education_num",
  "marital_status",
  "occupation",
  "relationship",
  "race",
  "sex",
  "capital_gain",
  "capital_loss",
  "hours_per_week",
  "native_country",
  "income"
)

# Preview the first few obs.
head(adult)

```

	age	workclass	fnlwgt	education	education_num	marital_status
1	39	State-gov	77516	Bachelors	13	Never-married
2	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse
3	38	Private	215646	HS-grad	9	Divorced
4	53	Private	234721	11th	7	Married-civ-spouse
5	28	Private	338409	Bachelors	13	Married-civ-spouse
6	37	Private	284582	Masters	14	Married-civ-spouse

	occupation	relationship	race	sex	capital_gain	capital_loss
1	Adm-clerical	Not-in-family	White	Male	2174	0
2	Exec-managerial	Husband	White	Male	0	0
3	Handlers-cleaners	Not-in-family	White	Male	0	0

4	Handlers-cleaners	Husband	Black	Male	0	0
5	Prof-specialty	Wife	Black	Female	0	0
6	Exec-managerial	Wife	White	Female	0	0
	hours_per_week	native_country	income			
1	40	United-States	<=50K			
2	13	United-States	<=50K			
3	40	United-States	<=50K			
4	40	United-States	<=50K			
5	40	Cuba	<=50K			
6	40	United-States	<=50K			

```
# check structure
str(adult)
```

```
'data.frame': 32561 obs. of 15 variables:
 $ age      : int  39 50 38 53 28 37 49 52 31 42 ...
 $ workclass : chr   "State-gov" "Self-emp-not-inc" "Private" "Private" ...
 $ fnlwgt    : int  77516 83311 215646 234721 338409 284582 160187 209642 45781 159449 .
 $ education : chr   "Bachelors" "Bachelors" "HS-grad" "11th" ...
 $ education_num : int  13 13 9 7 13 14 5 9 14 13 ...
 $ marital_status: chr   "Never-married" "Married-civ-spouse" "Divorced" "Married-
civ-spouse" ...
 $ occupation  : chr   "Adm-clerical" "Exec-managerial" "Handlers-cleaners" "Handlers-
cleaners" ...
 $ relationship : chr   "Not-in-family" "Husband" "Not-in-family" "Husband" ...
 $ race        : chr   "White" "White" "White" "Black" ...
 $ sex         : chr   "Male" "Male" "Male" "Male" ...
 $ capital_gain : int  2174 0 0 0 0 0 0 0 14084 5178 ...
 $ capital_loss : int  0 0 0 0 0 0 0 0 0 0 ...
 $ hours_per_week: int  40 13 40 40 40 40 16 45 50 40 ...
 $ native_country: chr   "United-States" "United-States" "United-States" "United-
States" ...
 $ income      : chr   "<=50K" "<=50K" "<=50K" "<=50K" ...
```

b.Clean and simplify the variable names.

```
names(adult) <- c(
  "age",
  "workclass",
  "weight",
```

```

"education",
"educ_num",
"marital",
"occupation",
"relationship",
"race",
"sex",
"cap_gain",
"cap_loss",
"hours",
"country",
"income"
)
names(adult)

```

```

[1] "age"          "workclass"    "weight"       "education"    "educ_num"
[6] "marital"      "occupation"   "relationship" "race"         "sex"
[11] "cap_gain"     "cap_loss"     "hours"        "country"      "income"

```

c.Examine missing values. Report the number of missing observations for each variable.

```

# create a table of missing observations in each variable
missing_table <- data.frame(
  Variable = names(adult),
  Missing = colSums(is.na(adult))
)

# display the number
missing_table

```

	Variable	Missing
age	age	0
workclass	workclass	1836
weight	weight	0
education	education	0
educ_num	educ_num	0
marital	marital	0
occupation	occupation	1843
relationship	relationship	0

race	race	0
sex	sex	0
cap_gain	cap_gain	0
cap_loss	cap_loss	0
hours	hours	0
country	country	583
income	income	0

Missing values are present in 'workclass', 'occupation', and 'country', with 1836, 1843, and 583 missing observations, respectively.

d. Restrict the sample to individuals working at least 10 hours per week. Confirm the sample size before and after filtering.

```
# check the sample size
n_before <- nrow(adult)
n_before
```

```
[1] 32561
```

```
# restrict the sample
adult <- adult[adult$hours >= 10, ]

# check the sample size after filtering
n_after <- nrow(adult)
n_after
```

```
[1] 32103
```

```
# create the table showing the sample size before and after
sample_size <- data.frame(
  Stage = c("Before", "After filtering"),
  Sample_Size = c(n_before, n_after)
)

# display the table
sample_size
```

	Stage	Sample_Size
1	Before	32561
2	After filtering	32103

The sample size decrease from 32561 to 32103 after restricting. A total of 458 observations were removed by this restriction.

e. Remove observations with missing values for occupation, workclass, or native country. Explain why.

```
# check the sample size
n_before_missing <- nrow(adult)

# remove obs. with missing values in occupation, workclass, or country
adult <- adult[
  complete.cases(adult[, c("occupation", "workclass", "country"))],
]

# check the sample size after removing missing values
n_after_missing <- nrow(adult)

# display the sample size
n_before_missing
```

```
[1] 32103
```

```
n_after_missing
```

```
[1] 29873
```

After removing observations with missing values in occupation, workclass, or country, the sample size decreased from 32,103 to 29,873. A total of 2,230 observations were removed. Observations with missing value were removed to ensure complete information for the variables used in the analysis.

“<https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/complete.cases>” helped me figure out a more efficient way to code my loop for problem 2 by doing ‘complete.cases’.

f. Eliminate any observations with: age less than 18, age greater than 90, education level identified as Preschool.

```
# check the sample size
n_before_filter <- nrow(adult)

# remove obs. with age below 18, age above 90, or education level equal to Preschool
adult <- adult[
  adult$age >= 18 &
  adult$age <= 90 &
  adult$education != "Preschool",
]

# check the sample size after filtering
n_after_filter <- nrow(adult)

# display
n_before_filter
```

```
[1] 29873
```

```
n_after_filter
```

```
[1] 29523
```

After removing observations with age less than 18, age greater than 90, or education level equal to Preschool, the sample size decreased from 29,873 to 29,523.

g. Report your final sample size.

```
final <- nrow(adult)
final
```

```
[1] 29523
```

The final sample size is 29,523 observations.

Q3:Happy Numbers

a. Write a function `nextHappy()` that given a positive integer, computes the next value in the happy-number sequence. Be sure to provide a reasonable error on an invalid input. Be sure to document your function.

```
# compute the next value in a happy number sequence
nextHappy <- function(n) {
  # check that n is a single positive integer
  if(length(n) != 1 || is.na(n) || n <= 0 || n != floor(n)) {
    stop("input must be a positive integer")
  }

  # split n into individual digits
  digits <- as.numeric(strsplit(as.character(n), "")[[1]])

  # return the sum of the digits
  sum(digits^2)
}

# demonstrate the function using the given examples
nextHappy(19)
```

```
[1] 82
```

```
nextHappy(82)
```

```
[1] 68
```

b. Create a function `happySequence()` that returns the sequence beginning at a given number. Use your `nextHappy()` function to perform the calculation. Be sure to provide a reasonable error on an invalid input. Be sure to document your function.

```
# generate a happy number sequence
happySequence <- function(n) {
  # check that n is a single positive integer
  if(length(n) != 1 || is.na(n) || n <= 0 || n != floor(n)) {
    stop("input must be a positive integer")
  }
}
```

```

}
# start the sequence
sequence <- n

# continue until the sequence reaches 1 or enters a cycle
while (tail(sequence, 1) != 1) {
  # calculate the next value
  next_value <- nextHappy(tail(sequence, 1))
  #check the next value has already appeared
  if(next_value %in% sequence) {
    return(list(
      sequence = sequence,
      isHappy = FALSE
    ))
  }
  # add the next value
  sequence <- c(sequence, next_value)
}
# return the sequence and TRUE if reaches 1
return(list(
  sequence = sequence,
  isHappy = TRUE
))
}

# demonstrate the function with a happy number
happySequence(19)

```

```

$sequence
[1] 19 82 68 100 1

```

```

$isHappy
[1] TRUE

```

```

happySequence(4)

```

```

$sequence
[1] 4 16 37 58 89 145 42 20

```

```

$isHappy
[1] FALSE

```

c.Among integers 1 through 1000: How many are happy? What percentage are happy? What starting value produces the longest happy sequence before terminating or cycling?

```
# happy number through 1 to 1000
results <- lapply(1:1000, happySequence)

# extract each starting value is happy
is_happy <- sapply(results, function(x) x$isHappy)

# count # of happy numbers
num_happy <- sum(is_happy)
num_happy
```

```
[1] 143
```

```
# calculate the percentage
per_happy <- mean(is_happy) * 100
per_happy
```

```
[1] 14.3
```

```
# calculate the sequence length for each happy number
sequence_lengths <- sapply(
  results,
  function(x) {
    if(x$isHappy) {
      length(x$sequence)
    } else{
      NA
    }
  }
)

# find the max happy sequence length
max_length <- max(sequence_lengths, na.rm = TRUE)
max_length
```

```
[1] 7
```

```
# find the starting values with the longest happy sequence
longest_start <- which(sequence_lengths == max_length)
longest_start
```

```
[1] 356 365 379 397 469 496 536 563 635 649 653 694 739 793 899 937 946 964 973
[20] 989 998
```

Among the integers from 1 to 1000, 143 are happy numbers, which is 14.3%. The longest happy sequence has length 7. Multiple starting values produce sequences of this maximum length, including 356, 365, 379, 397, 469, 496, 536, 563, 635, 649, 653, 694, 739, 793, 899, 937, 946, 964, 973, 989, and 998.